

© Brendan Choi 2021

B. Choi, *Introduction to Python Network Automation*

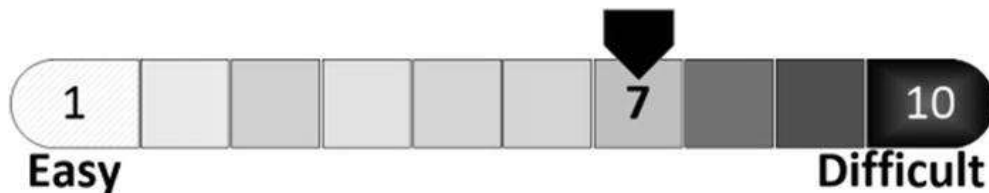
https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-6806-3_11

11. Cisco IOS Labs

Brendan Choi¹

(1) Sydney, NSW, Australia

This will be one of the shortest chapters in this book. You will test-drive your Linux virtual machines (VMs) to interact with the IOS router, R1, via Telnet. You will learn to create a new GNS3 project, run a quick file transfer testing lab using simple Cisco configurations, and then finally make a clone of the GNS3 project. You are making progressive steps toward Python network automation.



Cisco IOS and the Linux VM Lab

In this lab, you are going to connect your Linux virtual machines to GNS3 devices. In most production environments, running Python on Linux platforms is the norm to gain the numerous advantages of using Linux in an enterprise environment. So, merely understanding how to write Python code is insufficient to create a useful application to automate and streamline a network engineer's tasks. You need to have a deeper understanding of the whole IT ecosystem and know how different systems and technologies live harmoniously in an IT ecosystem. You have to be willing to step outside of your comfort zone and put personal effort into understanding other technologies in the IT ecosystem.

In this Cisco IOS and Linux VM example, we will configure a new topology to prepare for the next chapter. Figure **11-1** shows the topology that

you are going to configure. Note that the CentOS 8.1 and Ubuntu 20 LTS Linux servers are already connected to VMnet8 on the VMware Workstation in the topology shown. In GNS3, we will use representational icons to mark that they are on the 192.168.130.0/24 subnet. If you are using a different IP subnet for VMnet8, your IP address will be different.

The topology uses NAT-1 running on the host PC, connected to VMnet8 of VMware Workstation; it also acts as a DHCP server and a gateway to connect to the Internet. The host PC with IP address 7.7.7.1 is the cloud with the Microsoft loopback interface connecting to R1's F0/1 on the 7.7.7.0/24 network.

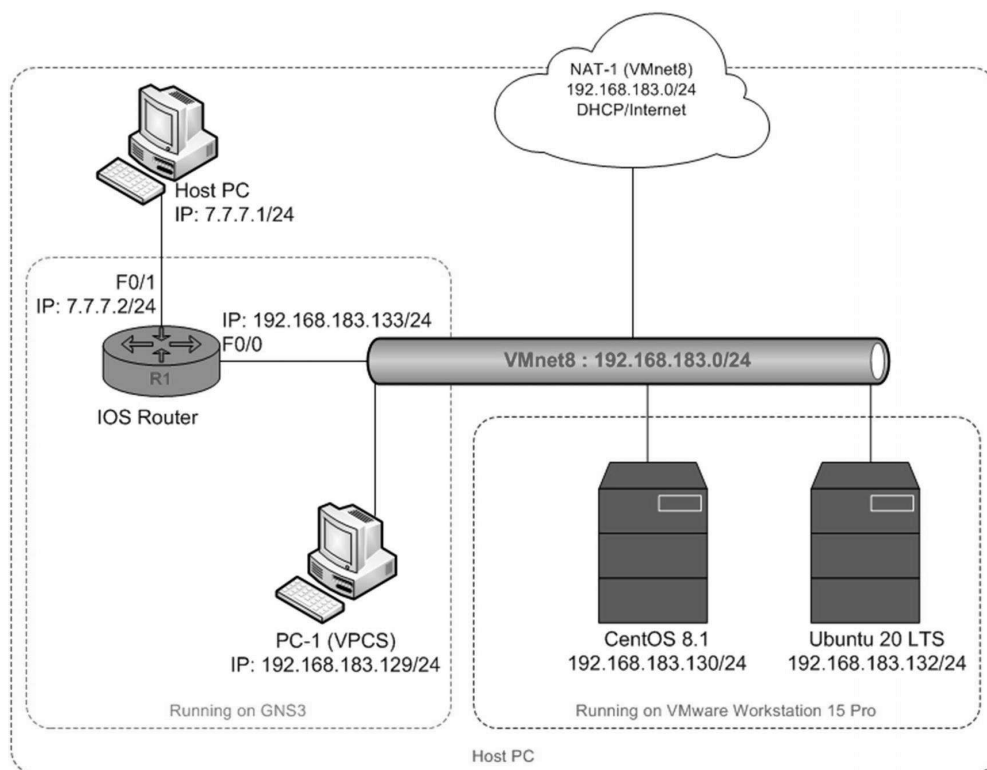


Figure 11-1. linuxvm2ios lab topology

Creating a New GNS3 Project for the Linux VM Lab

Let's create a new project to connect Linux VM servers to a new IOS router. The process of creating a new project is similar to the previous projects in Chapter 10. Let's create a new project to connect our Linux virtual machines to an IOS router.

#	Task
1	From the GNS3 main menu, select File ► New Blank Project to open a new project window.

Task

Give your new project a new name. The new project name is `linuxvm2ios` for this lab. You do not have to follow the same naming convention. Feel free to give your project a unique and meaningful name. See Figure [11-2](#).

2

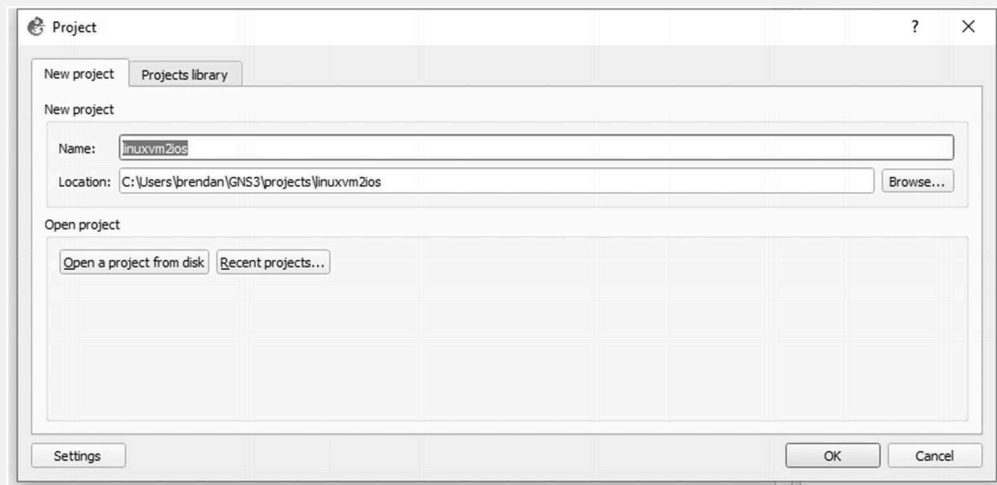


Figure 11-2. linuxvm2ios project, creating a new GNS3 project

- 3 Configure the GNS3 topology as shown in Figure [11-3](#). While building a new topology, refer to GNS3's topology and server summary window for more information. You will be able to see how each device is connected. When all devices are connected, start R1 and PC-1.
- The following points are connection references while connecting and configuring the topology on the GNS3 canvas:
- Run NAT-1 on the host server. (Use NAT from VMnet8 of Workstation.)
 - Ethernetswitch-1 is installed on the GNS3 VM server and only serves as a dummy switch to connect multiple devices.
 - PC-1 is a VPCS installed on GNS3. VPCS makes it easy to test communication between devices. This is similar to configuring dummy loopback interfaces on the router.
 - Host-PC marked with the Microsoft loopback is cloud-1, with a changed icon for aesthetics only.
 - Lastly, the CentOS 8.1 and Ubuntu 20.4 LTS servers are directly connected to VMnet8 on VMware Workstation and will not show up on the GNS3 topology. These server icons are also for aesthetic representation.

Task

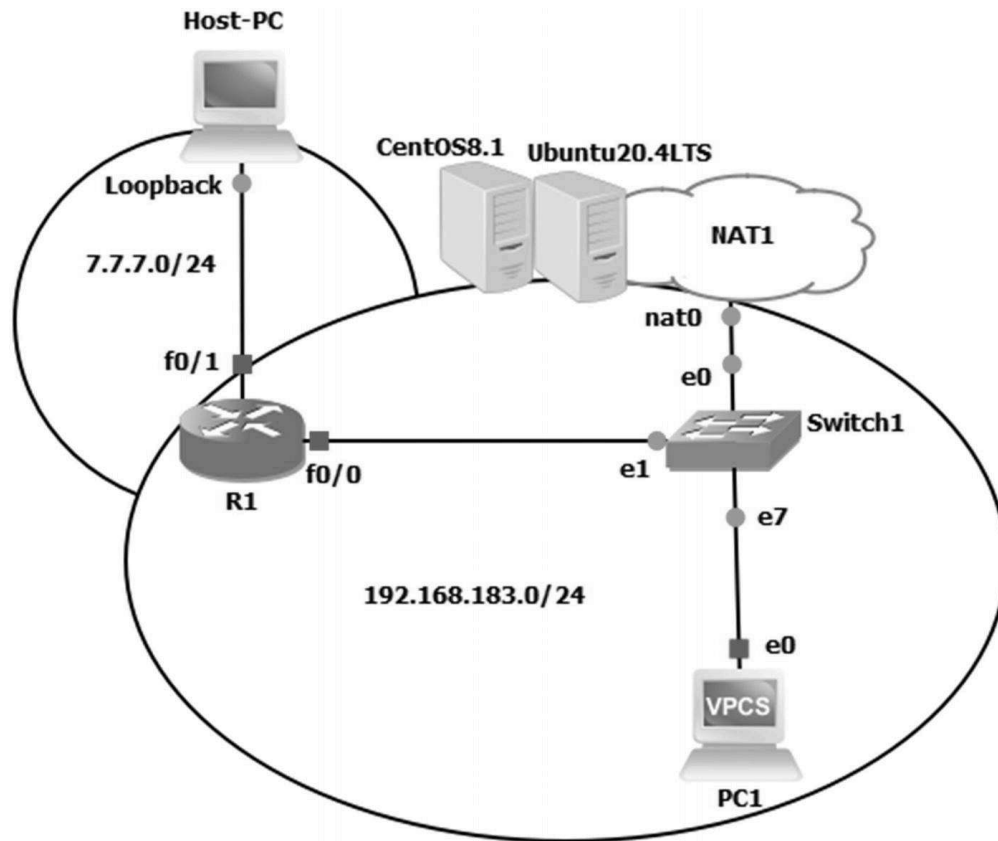


Figure 11-3. linuxvm2ios project, GNS3 topology connection

Note that in the topology in Figure [11-3](#), when you drag and drop NAT-1 and Host-PC (cloud-1), the network of VMnet8 should be used, so the host PC (pynetauto) should be selected. Your PC name will be different. See Figure [11-4](#).

Task

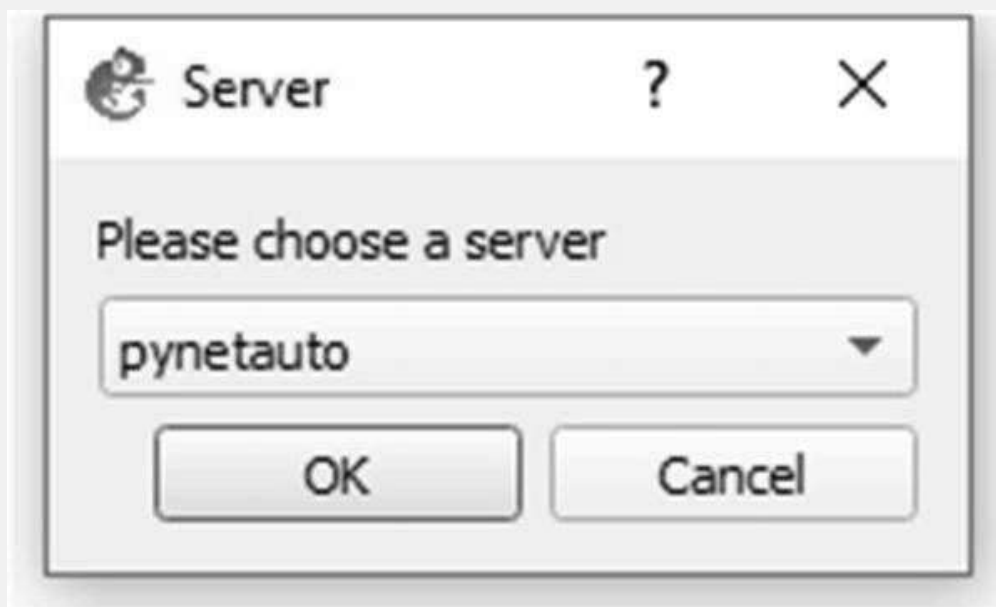


Figure 11-4. linuxvm2ios project, selecting the host server for NAT-1 and cloud-1

Use the Servers Summary window shown in Figure [11-5](#) as a reference point while configuring and running the GNS3 topology.

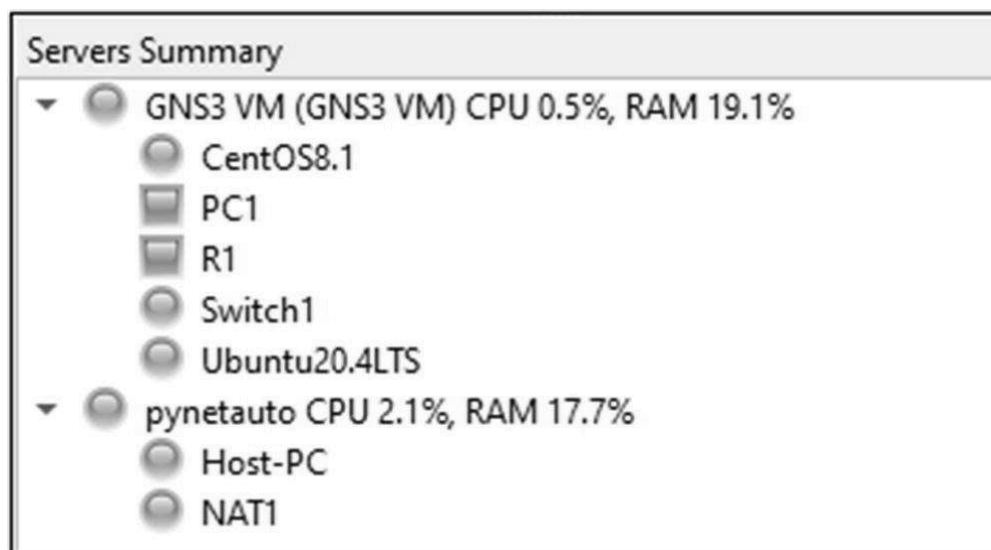


Figure 11-5. linuxvm2ios project, Servers Summary window

When you connect Ethernetswitch-1, a VPCS (PC-1), and an IOS router (R1), select GNS3 VM as the server, as shown in Figure [11-6](#).

Task

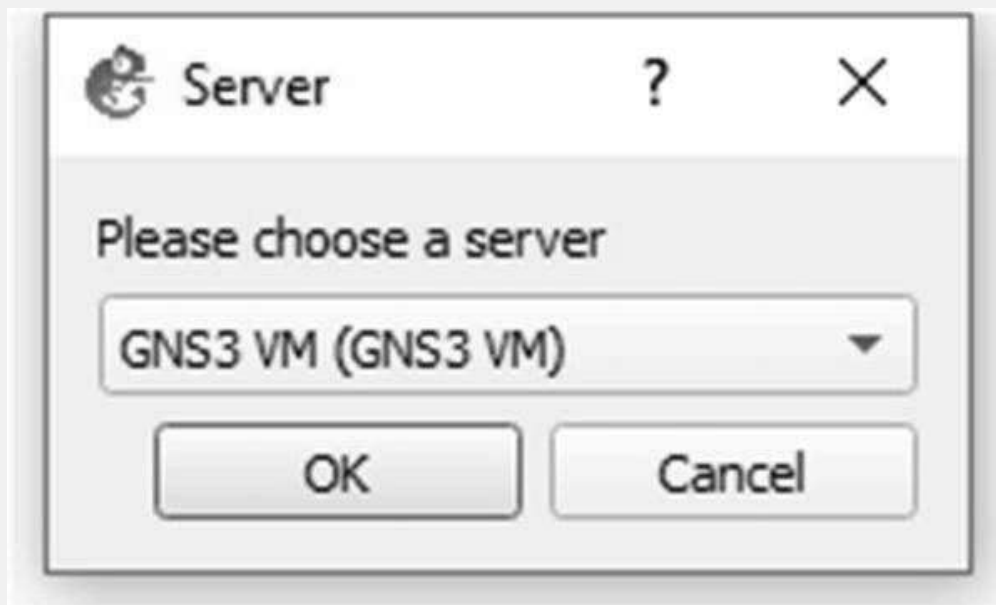


Figure 11-6. linuxvm2ios project, selecting the GNS3 VM server for all other devices

Refer to the Topology Summary window shown in Figure [11-7](#) to connect each device.

Task

Topology Summary	
Node	Console
CentOS8.1	none
Host-PC	none
Loopback <=> f0/1 R1	
NAT1	none
nat0 <=> e0 Switch1	
nat0 <=> e0 Switch1	
PC1	telnet 192.168.65.128:5002
e0 <=> e7 Switch1	
R1	telnet 192.168.65.128:5001
f0/0 <=> e1 Switch1	
f0/1 <=> Loopback Host-PC	
Switch1	none
e0 <=> nat0 NAT1	
e0 <=> nat0 NAT1	
e1 <=> f0/0 R1	
e7 <=> e0 PC1	
Ubuntu20.4LTS	none

Figure 11-7. linuxvm2ios project, topology summary

- Next, go to VMware Workstation's main user window and start the CentOS 8.1 VM. When the server boots up, log in as the `pynetauto` user or your username. See Figure [11-8](#).

Task

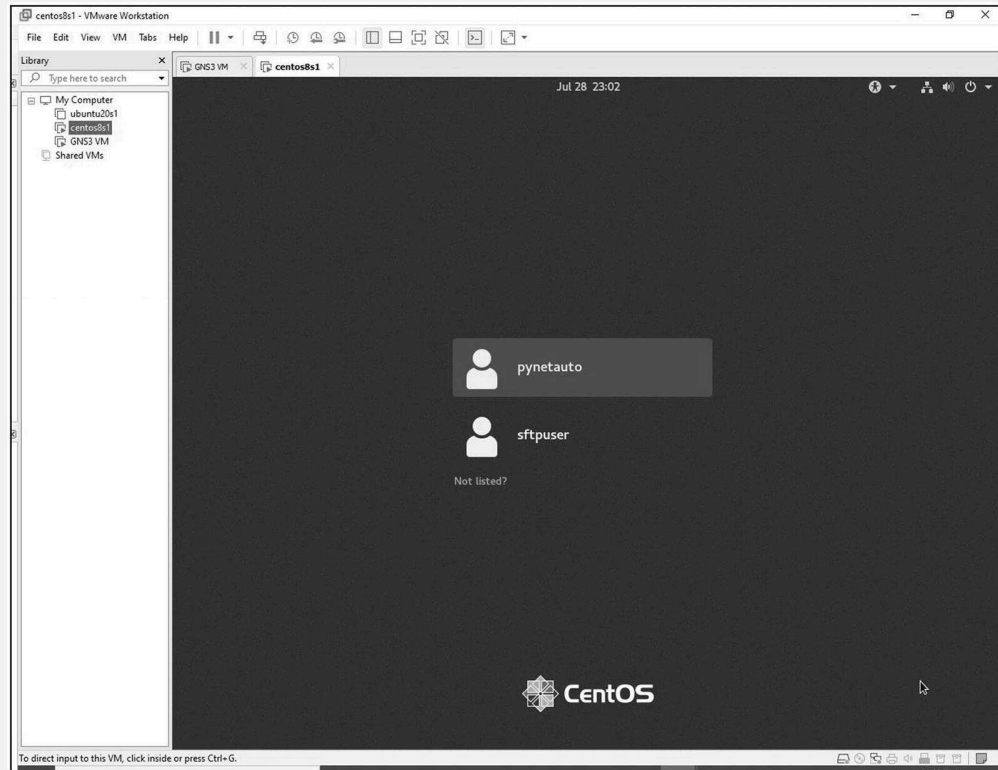


Figure 11-8. CentOS 8.1, login screen

- 5 Open a terminal on CentOS 8.1 and first check the IP address of this server. Use the `ifconfig ens160` command on the terminal to get or confirm the IP address of your server. You can alternatively SSH into your CentOS 8.1 server. An IP address from the 192.168.183.0/24 subnet should be assigned for this example, but if you are using a different subnet for VMnet8, you should receive a different IP address. In this example, an IP address of 192.168.183.130 has been allocated to the Ethernet 160 interface. Your IP address allocation may differ based on your virtual machine subnet.

```
[pynetauto@centos8s1 ~]$ ifconfig ens160
ens160: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.183.130 netmask 255.255.255.0 broadcast
192.168.183.255
    inet6 fe80::f49c:7ff4:fb3f:bf32 prefixlen 64 scopeid
0x20<link>
    ether 00:0c:29:85:a6:36 txqueuelen 1000 (Ethernet)
RX packets 87 bytes 10302 (10.0 KiB)
RX errors 0 dropped 0 overruns 0 frame 0
```

Task

```
TX packets 118 bytes 15005 (14.6 KiB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Still on the VMware workstation, start the Ubuntu 20 LTS server and find out the IP address of this server in the same manner. Issue the `ifconfig ens33` or `ifconfig` or `ip address` command.

An IP address of 192.168.183.132 has been assigned to the `ens33` interface. Your IP address allocation may differ based on your virtual machine subnet.

```
pynetauto@ubuntu20s1:~$ ifconfig ens33
```

6

```
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.183.132 netmask 255.255.255.0 broadcast
192.168.183.255
    inet6 fe80::20c:29ff:fef3:877c prefixlen 64 scopeid
0x20<link>
    ether 00:0c:29:f3:87:7c txqueuelen 1000 (Ethernet)
RX packets 222 bytes 18369 (18.3 KB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 442 bytes 41326 (41.3 KB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

- 7 Now on GNS3, start R1 and wait for 1 minute. Open the console of R1 and configure the IP domain lookup and FastEthernet 0/0 interface to get an IP address from VMnet8's DHCP server. Also, configure interface FastEthernet 0/1 with an IP address of 7.7.7.2 255.255.255.0; this interface connects to the Microsoft loopback of the host PC.

```
R1(config)# ip domain lookup
R1(config)# interface FastEthernet0/0
R1(config-if)# ip address dhcp
R1(config-if)# no shut
R1(config-if)#
*Mar  1 00:01:15.543: %LINK-3-UPDOWN: Interface
FastEthernet0/0, changed state to up
*Mar  1 00:01:16.543: %LINEPROTO-5-UPDOWN: Line protocol on
Interface FastEthernet0/0, changed state to up
```

Task

```
*Mar  1 00:01:26.003: %DHCP-6-ADDRESS_ASSIGN: Interface
FastEthernet0/0 assigned DHCP address 192.168.183.133, mask
255.255.255.0, hostname R1

# Confirm the interface status by running "show ip interface
brief"

R1#show ip interface brief

Interface                IP-Address      OK? Method
Status                  Protocol
FastEthernet0/0         192.168.183.133 YES
DHCP    up                up
FastEthernet0/1         unassigned      YES
unset  administratively down down
FastEthernet1/0         unassigned      YES
unset  administratively down down
```

Now configure interface f0/1 to be connected to the Microsoft loopback and ping 7.7.7.1.

```
R1#configure terminal

Enter configuration commands, one per line.  End with CNTL/Z.

R1(config)#interface FastEthernet 0/1

R1(config-if)#ip address 7.7.7.2 255.255.255.0

R1(config-if)#no shut

R1(config-if)#exit

R1#

*Mar  1 00:47:12.911: %SYS-5-CONFIG_I: Configured from console
by console

R1#ping 7.7.7.1

Type escape sequence to abort.

Sending 5, 100-byte ICMP Echos to 7.7.7.1, timeout is 2
seconds:

.!!!!

Success rate is 80 percent (4/5), round-trip min/avg/max =
8/12/20 ms
```

Task

From R1's console, ping the NAT-1 gateway (192.168.183.2), Centos 8.1 server (192.168.183.130), and Ubuntu 20 LTS server (192.168.183.132). If all the ICMP responses are received, you are on the right track. If you see the first packet dropped, that is the expected and normal ARP behavior, as explained earlier.

R1#**ping 192.168.183.2**

Type escape sequence to abort.

Sending 5, 100-byte ICMP Echos to 192.168.183.1, timeout is 2 seconds:

.!!!!

Success rate is 80 percent (4/5), round-trip min/avg/max = 8/13/24 ms

8

R1#**ping 192.168.183.130**

Type escape sequence to abort.

Sending 5, 100-byte ICMP Echos to 192.168.183.130, timeout is 2 seconds:

.!!!!

Success rate is 80 percent (4/5), round-trip min/avg/max = 8/13/20 ms

R1#**ping 192.168.183.132**

Type escape sequence to abort.

Sending 5, 100-byte ICMP Echos to 192.168.183.132, timeout is 2 seconds:

.!!!!

9 Similarly, start PC-1 (VPCS) on GNS3 and run the `ip dhcp` command to receive an IP address from the VMnet8 DHCP server. Use `show ip` to check the IP configuration.

PC1> **ip dhcp**

DDORA IP 192.168.183.129/24 GW 192.168.183.2

PC1> **show ip**

```
NAME           : PC1[1]
IP/MASK        : 192.168.183.129/24
GATEWAY        : 192.168.183.2
DNS            : 192.168.183.2
```

Task

```
DHCP SERVER : 192.168.183.254
DHCP LEASE  : 1719, 1800/900/1575
DOMAIN NAME : localdomain
MAC         : 00:50:79:66:68:00
LPORT       : 20010
RHOST:PORT  : 127.0.0.1:20011
MTU:        : 1500
```

PC1's interface received an IP address of 192.168.183.129. DDORA is not the name of a kid's cartoon character (*Dora the Explorer*). Here, DORA stands for Discovery, Offer, Request, Acknowledge, which is a communication message used when assigning an IP address through DHCP services between a server and a client.

- 10** Finally, from your host PC's Windows desktop, SSH into both Linux servers using PuTTY and perform similar ICMP tests. Enter the IP address of each server and save the sessions for easier access, as shown in Figure **11-9**.

Task

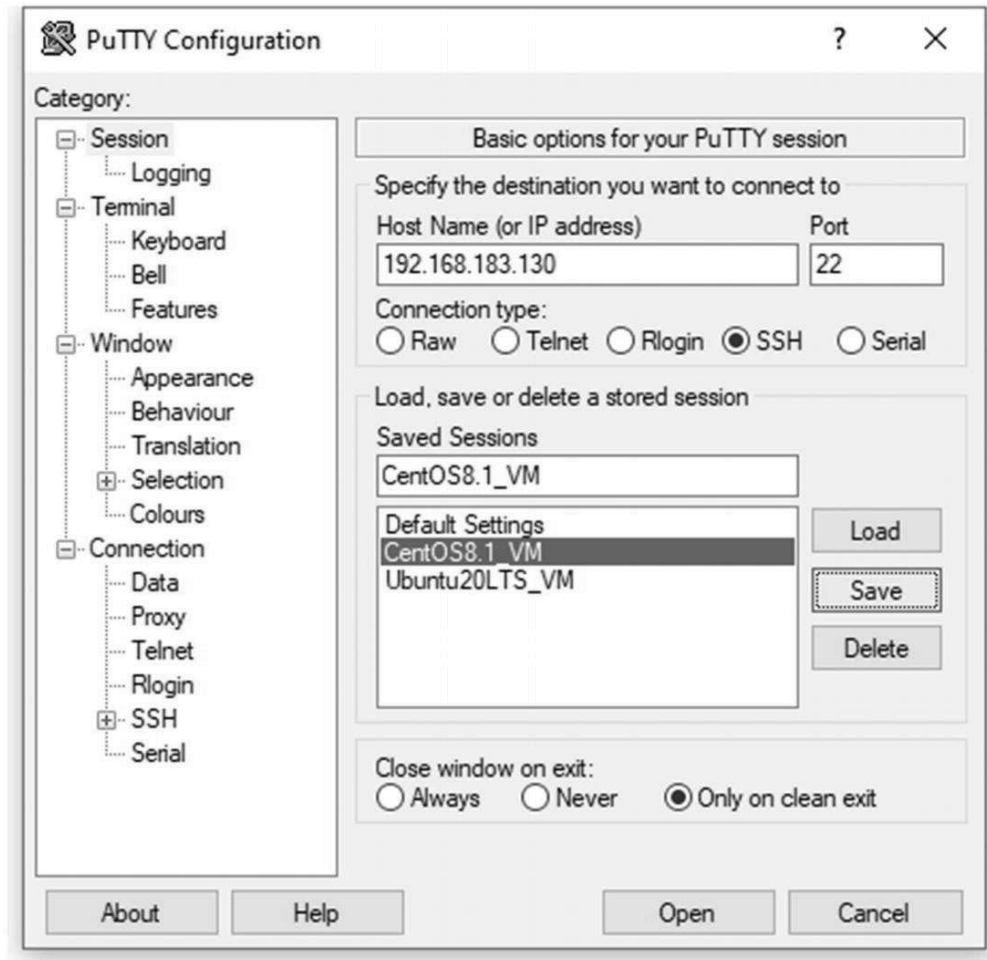


Figure 11-9. PuTTY, saving Linux sessions for convenience

If there is no communication problem, you are now ready to write some Python scripts on Linux servers and communicate to the Cisco IOS router. From your Linux servers, send ICMP messages to R1's `fa0/0` interface for a quick verification.

```
[pynetauto@centos8s1 ~]$ ping 192.168.183.133 -c 3
PING 192.168.183.133 (192.168.183.133) 56(84) bytes of data.
64 bytes from 192.168.183.133: icmp_seq=1 ttl=64 time=0.547 ms
64 bytes from 192.168.183.133: icmp_seq=2 ttl=64 time=0.318 ms
64 bytes from 192.168.183.133: icmp_seq=3 ttl=64 time=0.372 ms
--- 192.168.183.133 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 44ms
rtt min/avg/max/mdev = 0.318/0.412/0.547/0.099 ms
pynetauto@ubuntu20s1:~$ ping 192.168.183.133 -c 3
PING 192.168.183.133 (192.168.183.133) 56(84) bytes of data.
```

Task

```

64 bytes from 192.168.183.133: icmp_seq=1 ttl=255 time=5.92 ms
64 bytes from 192.168.183.133: icmp_seq=2 ttl=255 time=6.85 ms
64 bytes from 192.168.183.133: icmp_seq=3 ttl=255 time=5.70 ms
--- 192.168.183.133 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 5.699/6.158/6.851/0.498 ms

```

On R1, configure `enable password`, a username with level 15 privilege, and enable Telnet.

```

R1#configure terminal
R1(config)#enable password cisco123
R1(config)#username pynetauto privilege 15 password cisco123
R1(config)#line vty 0 15
R1(config-line)#login local
R1(config-line)#transport input telnet
R1(config-line)#no exec-timeout
R1(config-line)#end
R1#write memory

```

11

You can test the Telnet settings from the router itself, so on R1, run the `telnet 192.168.183.133` command. Also, use the `show user` command to check the logged-in user.

```

R1#telnet 192.168.183.133
Trying 192.168.183.133 ... Open
User Access Verification
Username: pynetauto
Password:
R1#show user

```

Line	User	Host(s)	Idle	Location
0 con 0		192.168.183.133	00:00:00	
* 98 vty 0	pynetauto	idle	00:00:00	

```

192.168.183.133
Interface      User                Mode                Idle      Peer
Address

```

Task

From your `ubuntu20s1` Linux server, Telnet to R1's `f0/0` interface (192.168.183.133) using PuTTY. Your allocated IP address may be different, so replace the IP address with your IP.

Run the `show user` command to check which users are logged in and the lines used for Telnet sessions. The asterisk (*) indicates access information from your active connection.

```
pynetauto@ubuntu20s1:~$ telnet 192.168.183.133
```

```
Trying 192.168.183.133...
```

```
Connected to 192.168.183.133.
```

```
Escape character is '^]'.
```

```
User Access Verification
```

```
Username: pynetauto
```

```
Password:
```

12 R1#show user

Line	User	Host(s)	Idle	Location
0 con 0		192.168.183.133	00:05:07	
98 vty 0	pynetauto	idle	00:05:07	
		192.168.183.133		
* 99 vty 1	pynetauto	idle	00:00:00	
		192.168.183.132		

Interface	User	Mode	Idle	Peer Address

Note, if you are using CentOS to Telnet into R1, then you may need to first install the Telnet client after connecting to the Internet.

```
[pynetauto@centos8s1 ~]$ sudo yum -y install telnet
```

or

```
[pynetauto@centos8s1 ~]$ sudo dnf -y install telnet
```

- 13** Now, from your Windows host PC, open another PuTTY session, and this time log in to R1 via `f0/1` (7.7.7.2). The * indicates access information from the current connection. Let's run `write memory` to save the current configuration of R1.

```
R1#show user
```

Line	User	Host(s)	Idle	Location
0 con 0		idle	00:05:43	

#	Task				
	98 vty 0	pynetauto	idle	00:00:40	
	192.168.183.133				
	99 vty 1	pynetauto	idle	00:01:10	
	192.168.183.132				
	*100 vty 2	pynetauto	idle	00:00:00	
	7.7.7.1				
	Interface	User	Mode	Idle	Peer
	Address				
	R1#write memory				

Now you have completed the communication and Telnet tests from all devices. Now you are ready to tackle another interesting IOS lab.

Uploading and Downloading Files to the GNS3 IOS Router from Linux VMs (File Transfer Testing Lab)

In this lab, you will add flash memory to GNS3’s IOS router, R1, and upload and download files from a TFTP server (CentOS8.1). You will write and run the script from the Ubuntu 20 LTS server.

Write a Python script to achieve the following tasks:

- Check the TFTP service is running correctly on CentOS8.1.
- Make a backup of R1’s running configuration to the TFTP server.
- Upload an IOS to the flash memory of R1 for a quick test.

#	Tasks
1	On GNS3, while creating the Cisco 3725 IOS router template, you have already added 64M to Cisco 3725’s PCMCIA disk0. If your router was not configured with a flash drive, first save the work and shut down the router. Then right-click, choose Configure, and move to the “Memories and disks” tab of R1. Add 64MB, cli Apply, and then click the OK button. Once the flash memory has been configured, start R1 again. See Figure 11-10.

Tasks

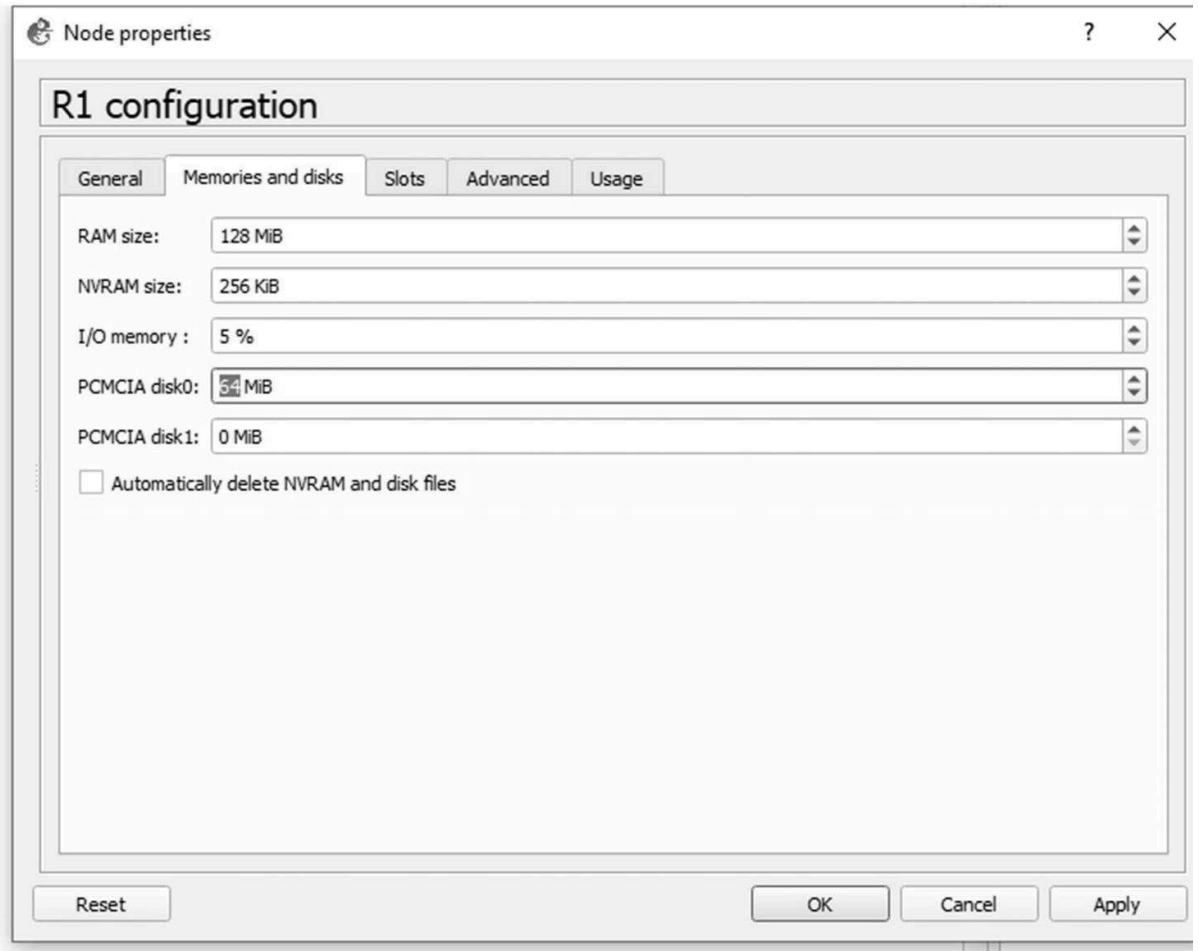


Figure 11-10. R1, checking flash memory allocation

- 2 Remember that in 8.3.3 we configured a TFTP server on CentOS8.1? Go to the PuTTY session of CentOS8.1 and now confirm the status of TFTP services. TFTP was configured during the IP Services installation on CentOS8.1 Linux.

Use the following commands to check for TFTP services running on CentOS8.1.

<code>sudo systemctl status tftp</code>	Checks TFTP service status
<code>sudo systemctl enable tftp</code>	Enables TFTP services on system startup
<code>sudo systemctl start tftp</code>	Starts TFTP services

If the TFTP service on CentOS 8.1 is running correctly, it should display active (running) in green. If you have issues with TFTP, use the `enable` and `start` commands shown earlier to start the service on this server. If CentOS asks for the `sudo` user password, enter your password. In 4.5.3, we have configured a default folder called `/var/tftpd` for the TFTP service to share files from and to this folder.

Tasks

```
[pynetauto@localhost ~]$ sudo systemctl enable tftp
[sudo] password for pynetauto: *****
[pynetauto@localhost ~]$ sudo systemctl start tftp
[pynetauto@localhost ~]$ sudo systemctl status tftp
● tftp.service - Tftp Server
```

```
Loaded: loaded (/usr/lib/systemd/system/tftp.service; indirect; vendor
prese>
```

```
Active: active (running) since Fri 2021-01-08 23:43:05 AEDT; 4s ago
```

```
Docs: man:in.tftpd
```

```
Main PID: 4237 (in.tftpd)
```

```
Tasks: 1 (limit: 11166)
```

```
Memory: 328.0K
```

```
CGroup: /system.slice/tftp.service
```

```
└─4237 /usr/sbin/in.tftpd -s /var/lib/tftpboot
```

```
Jan 08 23:43:05 localhost systemd[1]: Started Tftp Server.
```

```
lines 1-11/11 (END)
```

- 3** On your Ubuntu server (primary Python server), reuse the Telnet sample code from `telnet_script1` from Chapter 10, and let's rewrite the Telnet script to make a backup of R1's running configuration. I am going to rename the script as `backup_config.py` and modify the code. The sections of code that are modified are highlighted. Go to Ubuntu 20 LTS Python server and create the following code using the nano text editor:

```
pynetauto@ubuntu20s1:~$ pwd
/home/pynetauto
pynetauto@ubuntu20s1:~$ mkdir ch11
pynetauto@ubuntu20s1:~$ cd ch11
pynetauto@ubuntu20s1:~/ch11$ nano backup_config.py
pynetauto@ubuntu20s1:~/ch11$ ls
backup_config.py
backup_config.py
import getpass
import telnetlib
HOST = "192.168.183.133"
```

Tasks

```

user = input("Enter your telnet username: ")
password = getpass.getpass()
tn = telnetlib.Telnet(HOST)
tn.read_until(b"Username: ")
tn.write(user.encode('ascii') + b"\n")
if password:
    tn.read_until(b"Password: ")
    tn.write(password.encode('ascii') + b"\n")
tn.write(b"copy running-config tftp://192.168.183.130/running-config\n") #
<<< Copy command
tn.write(b"\n") #<<< Enter key action
tn.write(b"\n") #<<< Enter key action
tn.write(b"exit\n") #<<< end session
print(tn.read_all().decode('ascii'))

```

Before running the previous script, go to CentOS8s1 and confirm the files in the `/var/tftpd` directory . You can see that there are only two transfer test files from 8.3.3

```
[pynetauto@centos8s1 ~]$ pwd
```

4 /home/pynetauto

```
[pynetauto@centos8s1 ~]$ cd /var/tftpd
```

```
[pynetauto@centos8s1 tftpd]$ ls
```

```
transfer_file01
```

```
transfer_file77
```

5 Now from Ubuntu 20 LTS Python server, run the `python3 backup_config.py` command Follow the prompt and check the returned output.

Enter your network admin ID and password to run the script. As the running configuration gets copied to the TFTP server, you should see a similar message to the following:

```
pynetauto@ubuntu20s1:~/ch11$ python3 backup_config.py
```

```
Enter your telnet username: pynetauto
```

```
Password: *****
```

```
R1#copy running-config tftp://192.168.183.130/running-config
```

```
Address or name of remote host [192.168.183.130]?
```

```
Destination filename [running-config]?
```

```
!!
```

```
1155 bytes copied in 2.168 secs (533 bytes/sec)
```

Tasks

```
R1#exit
```

Now go back to the CentOS8s1 server and recheck the directory (/var/tftpddir). You should now see that the `running-config` file was saved on the TFTP server.

```
6 [pynetauto@centos8s1 tftpd-dir]$ ls
running-
config          transfer_file01          transfer_file
```

Now to upload an IOS image to GNS3's IOS router's flash, you first have to format the flash. Open R1's console and follow these instructions:

```
7 R1#show flash
No files on device
134211584 bytes available (2198889009152 bytes used)

*Mar  1 02:53:58.163: %PCMCIAFS-5-DIBERR: PCMCIA disk 0 is formatted from
different router or PC. A format in this router is required before an image
can be booted from this device

R1#show file system
```

File Systems:

Size(b)	Free(b)	Type	Flags	Prefixes
-	-	opaque	rw	archive:
-	-	opaque	rw	system:
-	-	opaque	rw	tmpsys:
57336	54057	nvram	rw	nvram:
-	-	opaque	rw	null:
-	-	network	rw	tftp:
* 2199023220736	134211584	disk	rw	flash:
-	-	flash	rw	slot0:
-	-	opaque	wo	syslog:
-	-	opaque	rw	xmodem:
-	-	opaque	rw	ymodem:
-	-	network	rw	rcp:
-	-	network	rw	pram:
-	-	network	rw	ftp:

Tasks

```
- network rw http:
- network rw scp:
- opaque ro tar:
- network rw https:
- opaque ro cns:
```

R1#**format flash:**

Format operation may take a while. Continue? [**confirm**] <<< Press the [Enter] key

Format operation will destroy all data in "flash:". Continue? [**confirm**] < Press the [Enter] key

Writing Monlib sectors....

Monlib write complete

Format: All system sectors written. OK...

Format: Total sectors in formatted partition: 131040

Format: Total bytes in formatted partition: 67092480

Format: Operation completed successfully.

Format of flash: complete

R1#**show flash:**

No files on device

66936832 bytes available (0 bytes used)

You can use the FileZilla client for either Windows or WinSCP to upload any IOS file or any file for file transfer using the Python `telnetlib` module. If you do not have WinSCP on your Windows host PC, go to the following download site and install one:

URL: <https://winscp.net/eng/download.php> (WinSCP download)

URL: <https://filezilla-project.org/download.php?platform=win64> (FileZilla client for Windows)

8

After installing WinSCP and launching the application, use the following settings to connect to the CentOS8.1 server via port 22 (SSH). See Figure **11-11**.

File protocol: **SCP**

Host name: **192.168.183.130 (Your CentOS 8.1 IP address)**

Username: **pynetauto (Your CentOS 8.1 username)**

Password : ******* (Your CentOS 8.1 password)**

Tasks

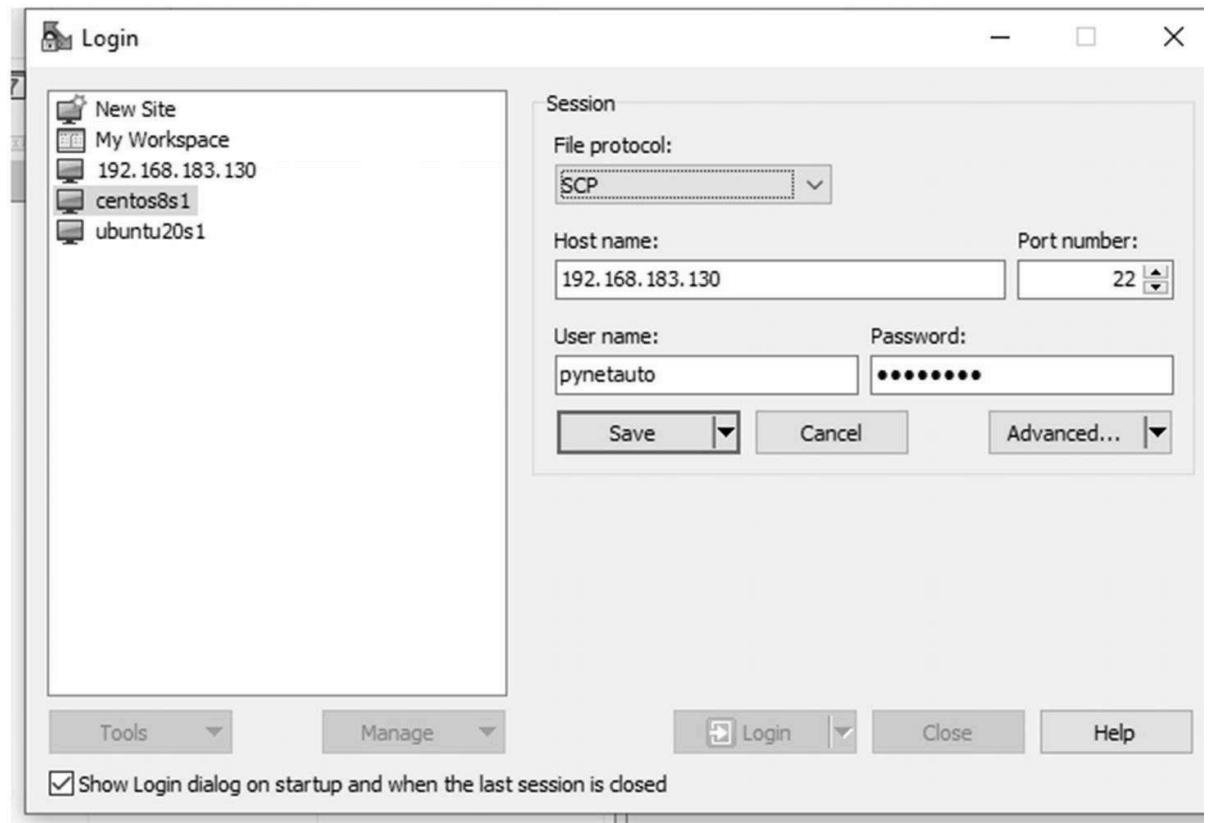


Figure 11-11. WinSCP, connection to TFTP server (CentOS8.1)

- 9 When you accept the SSH key and log in, drill down to the `/var/tftpddir/` directory of your TFTP (CentOS8.1) server and drag and drop an IOS file for file sharing. You can also use another file in place of the IOS file to save time as you will be testing the file transfer concept only using Python's `telnetlib` module. In this example, I am using the old IOS file named `c3725-adventerprisek9-mz.124-15.T14.bin`, but you can choose to use any file here for this test. See Figure [11-12](#).

Tasks

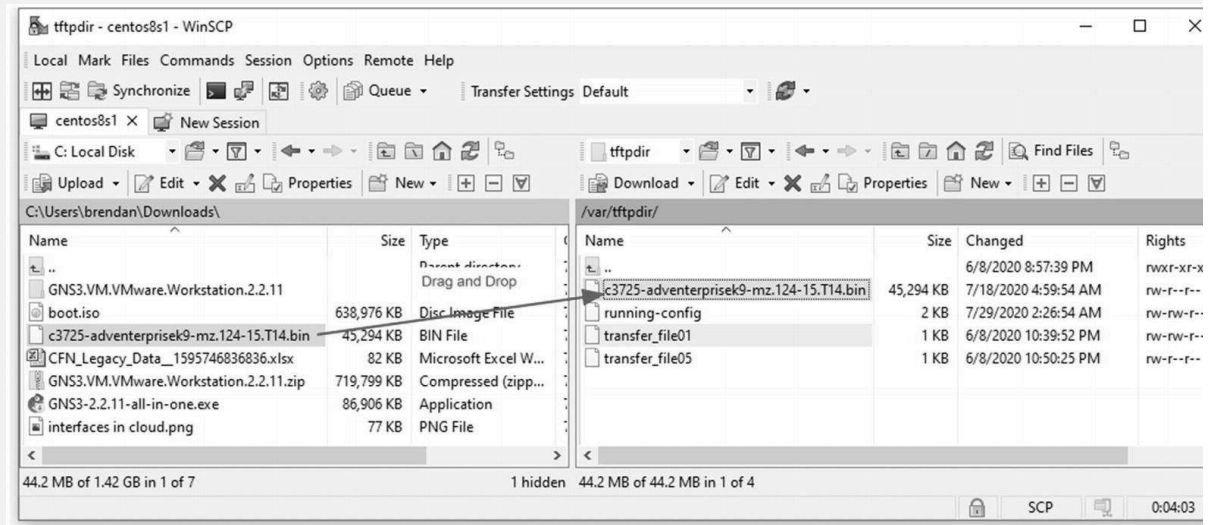


Figure 11-12. WinSCP, copying an IOS or any file transfer test file

10

On R1, the flash drive has been formatted in previous steps, and there is no need to check if any files exist on the flash.

Now on the Ubuntu 20 LTS server, copy `backup_config.py` and save it as `upload_ios.py`

```
pynetauto@ubuntu20s1:~/ch11$ ls
```

```
backup_config.py
```

```
pynetauto@ubuntu20s1:~/ch11$ cp backup_config.py upload_ios.py
```

11

```
pynetauto@ubuntu20s1:~/ch11$ nano upload_ios.py
```

Then modify the Python code so it looks like the following code. The updated parts are highlighted. You can type in the script from scratch, but it is easier to reuse the previous script and make the required modifications. Also, the script is available for download from the official download site.

`upload_ios.py`

```
import getpass

import telnetlib

HOST = "192.168.183.133"

user = input("Enter your telnet username: ")

password = getpass.getpass()

tn = telnetlib.Telnet(HOST)

tn.read_until(b"Username: ")

tn.write(user.encode('ascii') + b"\n")
```

Tasks

```

if password :
    tn.read_until(b"Password: ")
    tn.write(password.encode('ascii') + b"\n")

tn.write(b" copy tftp://192.168.183.130/c3725-adventerprisek9-mz.124-
15.T14.bin flash:c3725-adventerprisek9-mz.124-15.T14.bin\n") #<<< Run cop
command

tn.write(b"\n") #<<< Enter key action
tn.write(b"exit\n") #<<< end session
print(tn.read_all().decode('ascii'))

```

Run the Python code. It looks like the application is hanging, but in fact, it is transferring the file from the TFTP server to R1's flash memory. TFTP is a slow protocol, and it takes a long time to transfer this 45MB file. In your practice, you can create any dummy .txt file that is small to save time.

12

```

pynetauto@ubuntu20s1:~/ch11$ python3 upload_ios.py
Enter your telnet username: pynetauto
Password: *****
|

```

After running the previous script, go back to R1's console and use the `show users` command for the vty session for Telnet and use the `dir` or `show flash:` command to check that the file transfer is taking place.

R1#**show flash:**

```
-#- --length-- -----date/time----- path
```

13

```

1      1470464 Mar 01 2002 01:02:28 c3725-adventerprisek9-mz.124-15.T14.bi
65466368 bytes available (1470464 bytes used)

```

R1#**show flash:**

```
-#- --length-- -----date/time----- path
```

```

1      2936832 Mar 01 2002 01:02:58 c3725-adventerprisek9-mz.124-15.T14.bi
64000000 bytes available (2936832 bytes used)

```

- 14 If you are uploading an IOS file as shown in this example, it will take a while for the file transfer to complete via TFTP. The TFTP protocol uses the UDP port for file transfer but is slow file transfer protocol in both lab and production. You are only uploading a file to learn the concept of TFTP file transfer using a Python 3 script. You cannot update the boot system configuration and reboot the GNS3 IOS router with the new IOS image; this is one

Tasks

of the downsides of using IOS on GNS3. However, using Cisco CSR1000v or Nexus 9000v on VMware Workstation, you can simulate the actual IOS XE upgrade process, including booting into the newly upgraded IOS. In the next chapter, we'll use Cisco CSR1000v on VMware Workstation and emulate the full IOS upgrade process.

Once the file size reaches the expected file size, you know that the script and the file transfer have completed successfully.

```
R1#dir
Directory of flash:/
1  -rw-      46380064   Mar 1 2002 01:22:04 +00:00  c3725-adventerprisek9-
mz.124-15.T14.bin
66936832 bytes total (20553728 bytes free)
R1#show flash:
-#- --length-- -----date/time----- path
1      46380064 Mar 01 2002 01:22:04 c3725-adventerprisek9-mz.124-15.T14.bi
20553728 bytes available (46383104 bytes used)
```

Because of the slow uploading time and issues with Python's `telnetlib` library, your Ubuntu Python server console may still look like it is uploading the IOS. Press Ctrl+Z to exit this state to complete this lab.

```
pynetauto@ubuntu20s1:~/ch11$ python3 upload_ios.py
```

15 Enter your telnet username: pynetauto

Password:

^Z

```
[1]+  Stopped                  python3 upload_ios.py
```

```
pynetauto@ubuntu20s1:~/ch11$
```

You have used the Telnet and TFTP services to upload files to R1's flash. A large part of the network engineer's work involves IOS patch management involving uploading IOS to many routers and switches. Imagine yourself writing a piece of code, which can upload multiple IOS files to hundreds of Cisco routers and switches during the night while you are safe and sound asleep. In reality, this was one of the very first tools I had to create for regular patching of my clients' network devices.

Copying (Cloning) a GNS3 Project

To prepare for the next chapter, let's copy and clone the current project. There are about three different ways you can make a carbon copy of a GNS3 project. The first method is using the "Save project as..." feature of the GNS3 menu, the second method is to export it as a portable project and re-import it on another host PC, and the last method is to use the "Export config" method to export your devices' configuration and manually clone your GNS3 project. These methods are mentioned in order of increasing difficulty. Also, note that if you use the first method, your existing lab's configuration will be moved to the newly saved `Project` folder. You will lose the existing configurations of your devices' running configuration. The best way to clone a GNS3 project is by using the third method, which preserves your existing lab status and carries the same settings to the new project. Last, but most importantly, if you want to make a full backup of your GNS3 project at any time, you can copy the whole project folder under `C:\Users\[your_name]\GNS3\project`.

Since we will not be using the `linuxvm2ios` project beyond this point of the book, let's see how to clone the GNS3 project using a copy method. In the later chapter, another method will be demonstrated to give you another cloning option.

Task

- 1 Make sure all devices in the GNS3 project are powered off. To save the existing `linuxvm2ios` project to a new project name, first open GNS3's main user window, go to File, and select "Save project as." See Figure **11-13**.

Task

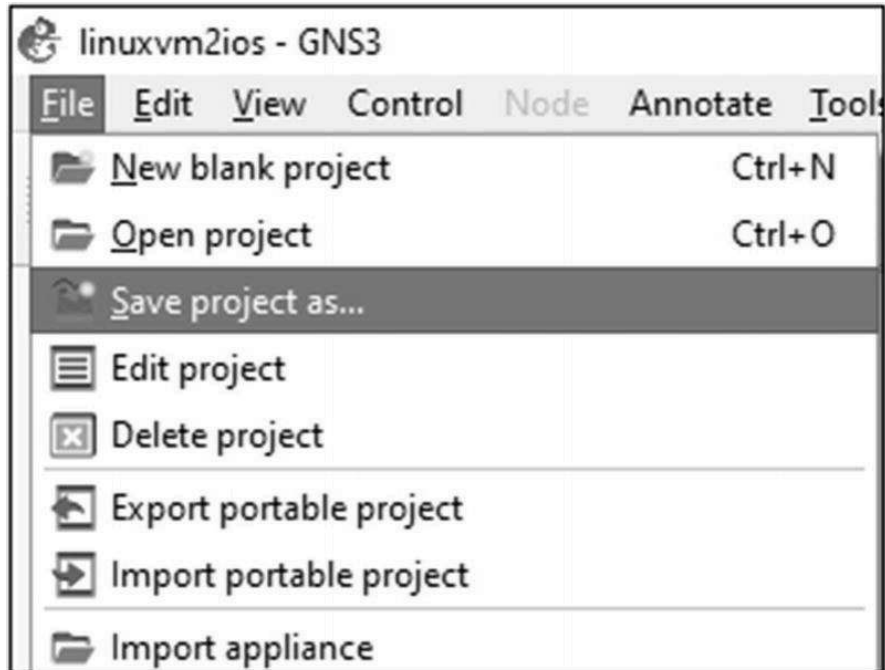


Figure 11-13. GNS3, “Project as...” menu

The GNS3 projects are saved under the `C:\Users\[your_name]\GNS3\project` folder. Now, give a new name to your new project for Chapter [12](#). This lab will be used across subsequent labs. See [Figure 11-14](#).

GNS3 project folder location: `C:\Users\brendan\GNS3\projects`

2

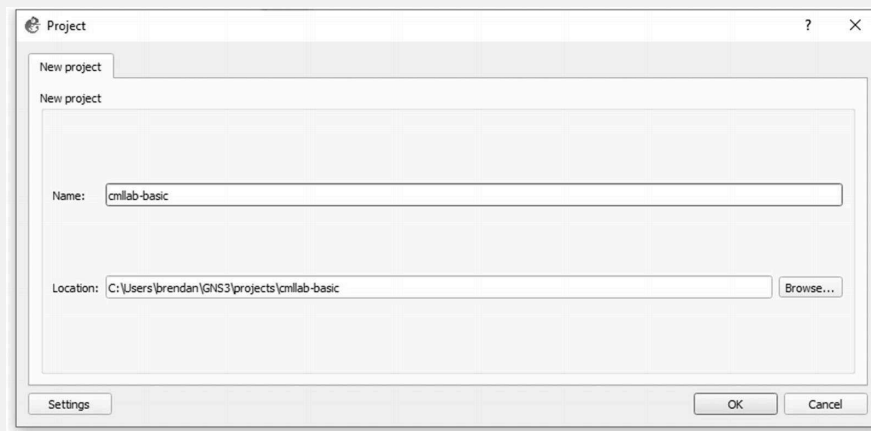


Figure 11-14. GNS3, cloning the existing linuxvm2ios project to cmllab-basic

Task

After you saved the new project, go to the `Project` folder (`C:\Users\[your_name]\GNS3\projects`) and check the original and new project folders' file sizes. The original project folder size will be around 24KB in size, whereas the new project folder size will be over 2.1MB. Open the folder and check the folder and file contents. See Figure **11-15**.

3

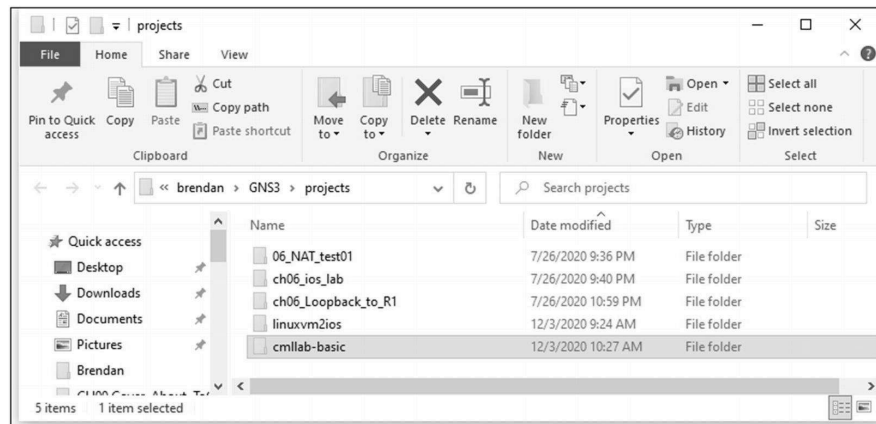


Figure 11-15. Host PC, confirming the new project folder

Task

Now open the new project and run it. If you have saved the project correctly, it should run smoothly with no errors. Now you are ready for some CML-PE labs in Chapter [12](#). See Figure [11-16](#).

4

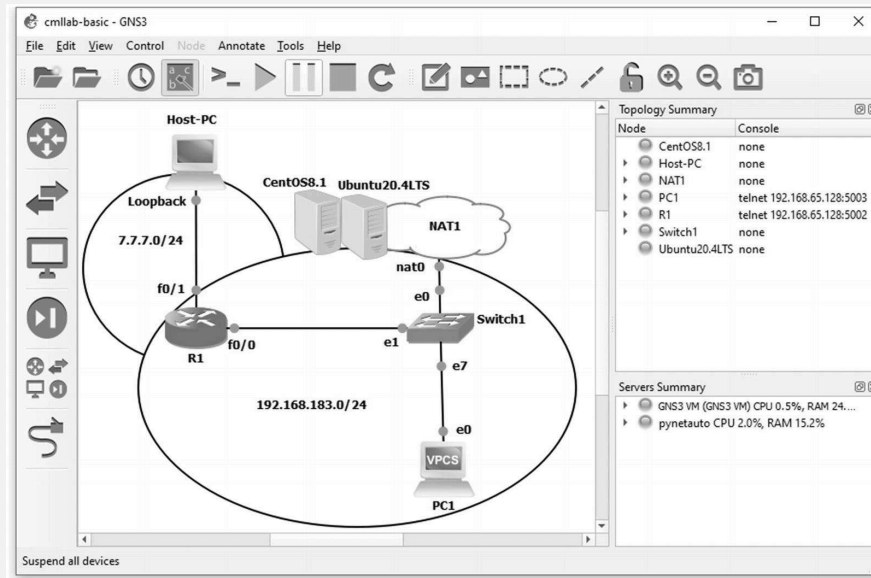


Figure 11-16. GNS3, running the cmllab-basic project

Summary

In this chapter, you tested Linux virtual machines' interaction with the IOS router, R1, via Telnet. You learned how to create a GNS3 project, ran a quick file transfer testing lab using simple Cisco configurations, and then finally learned to make a clone of the GNS3 project. You are now well positioned to tackle other labs in Chapters [12](#) to [18](#). In Chapter [12](#), we will use many of the skills acquired from previous chapters to build L3 routing and L2 switching labs using Cisco CML (VIRL) images integrated into GNS3. We will take some baby steps into Python network automation using the `telnetlib` module in Chapter [13](#) and then learn to control Cisco VIRL devices using the SSH protocol in Chapter [14](#). Python scripts will run on Linux VMs to control Cisco VIRL devices via the Telnet and SSH protocols using the `telnetlib`, `paramiko`, and `netmiko` libraries.

Table of contents collapsed